POLITECHNIKA WROCŁAWSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

PROJEKTOWANIE EFEKTYWNE ALGORTYMÓW

TSP – Problem Komiwojażera

Zadanie 4

Grupa: Wtorek 9:15

Autor:

Karolina Obiedzińska

Indeks: 273034

Prowadzący:

dr inż. Marcin Łopuszyński

Spis treści

1. Wstęp 3
1.1. Opis algorytmu 3
1.2. Omówienie elementów algorytmu 3
1.2.1 Reprezentacja chromosomów 3
1.2.2 Inicjalizacja populacji 4
1.2.3 Funkcja oceny 4
1.2.4 Operatory genetyczne 4
1.2.4.1 Selekcja 4
1.2.4.2 Krzyżowanie 4
1.2.4.3 Mutacja 5
1.2.5 Strategia wymiany pokoleń 5
1.2.6 Kryterium zatrzymania 5
1.2.7 Parametry sterujące 5
2. Opis klas 5
3. Wyniki eksperymentu 10
3.1. Badanie wpływu wielkości populacji 10
3.2. Badanie wpływu wielkości współczynnika mutacji 11
3.3. Badanie wpływu wielkości współczynnika krzyżowania 12
4. Wnioski 13
5. Kod źródłowy 13
Bibliografia: 13

1. Wstęp

1.1. Opis algorytmu

Algorytm genetyczny

- to zaawansowana technika obliczeniowa inspirowana procesami ewolucyjnymi zachodzącymi w przyrodzie, wykorzystywana w dziedzinie sztucznej inteligencji i optymalizacji.[1] Jest to heurystyczna metoda przeszukiwania przestrzeni alternatywnych rozwiązań w celu znalezienia optymalnego lub bliskiego optymalnemu rozwiązania dla złożonych problemów.[2]

Kluczowe cechy algorytmu genetycznego:

- Inspiracja procesami doboru naturalnego i procesami ewolucji
- Populacja rozwiązań
- Funkcja oceny
- Operatory genetyczne

Zasada działania[3]

1. Losowana jest pewna populacja początkowa.
2. Populacja poddawana jest ocenie (selekcja). Najlepiej przystosowane osobniki bior ą udział w reprodukcji.
3. Genotypy wybranych osobników poddawane są operatorom ewolucyjnym:
   a) krzyżowania (odpowiednie połączenie genotypów rodziców),
   b) mutacji (wprowadzenia do genotypów drobnych losowych zmian).
4. Utworzenie kolejnego pokolenia (nowej populacji). Jeżeli nie znaleziono dostatecznie dobrego rozwiązania, to powrót do punktu 2. W przeciwnym wypadku uzyskujemy wynik.

1.2. Omówienie elementów algorytmu

1.2.1 Reprezentacja chromosomów

- sposób zakodowania rozwiązania problemu w postaci chromosomu (genotypu). W przypadku problemu komiwojażera chromosom najczęściej reprezentowany jest jako permutacja liczb całkowitych, gdzie każda liczba oznacza konkretne miasto.

---

[1] https://kotrak.com/pl/blog/algorytmy-genetyczne-w-systemach-it-i-przemysle/
[2] https://pl.wikipedia.org/wiki/Algorytm_genetyczny
[3] http://staff.iiar.pwr.wroc.pl/dariusz.banasiak/si/SI_wyklad10.pdf

Przykładowo, dla 5 miast chromosom może wyglądać tak: [2, 4, 1, 5, 3], co oznacza kolejność odwiedzania miast:

miasto 2 **–>** miasto 4 **–>** miasto 1 **–>** miasto 5 **–>** miasto 3 **–>** powrót do miasta 2.

1.2.2 Inicjalizacja populacji

- tworzenie początkowej populacji rozwiązań. Populacja początkowa jest zwykle generowana losowo poprzez tworzenie losowych permutacji miast. Można też zastosować proste heurystyki, np. algorytm najbliższego sąsiada, aby uzyskać lepsze rozwiązania początkowe.

1.2.3 Funkcja oceny

- funkcja określająca jakość danego rozwiązania. Dla problemu komiwojażera funkcją oceny jest całkowita długość trasy, którą należy zminimalizować. Oblicza się ją sumując odległości między kolejnymi miastami w trasie, uwzględniając powrót do miasta początkowego.

1.2.4 Operatory genetyczne[4]

1.2.4.1 Selekcja

- polega na wyborze najlepiej przystosowanych osobników do grupy rozrodczej, których genotypy zostaną przekazane do kolejnych pokoleń. Kluczowe jest właściwe określenie proporcji tej grupy względem całej populacji. Zbyt mały udział najlepszych osobników może prowadzić do utraty różnorodności genetycznej i osłabienia populacji, natomiast zbyt duży udział słabych osobników obniża jakość rozwiązań, utrudniając osiągnięcie optymalnych wyników.

1.2.4.2 Krzyżowanie

- polega na tworzeniu nowej populacji poprzez wymianę fragmentów genotypów między osobnikami z grupy rozrodczej. Proces ten odbywa się przez rozcięcie chromosomów rodziców w losowym punkcie, po czym połowa chromosomu trafia do jednego potomka, a druga połowa do drugiego. W ten sposób generowane są nowe kombinacje genów, co wspiera eksplorację przestrzeni rozwiązań.

- **Operator krzyżowania PMX** (Partially Matched Crossover, Goldberg i Lingle, 1985):

Jest to operator, który działa w następujących krokach:

1. Losowo wybierane są dwa punkty krzyżowania
2. Fragment między tymi punktami jest kopiowany bezpośrednio z pierwszego rodzica do potomka
3. Algorytm analizuje analogiczny segment w drugim rodzicu i znajduje elementy, które nie zostały jeszcze skopiowane do potomka
4. Dla każdego takiego elementu i określany jest element j, który został skopiowany z pierwszego rodzica na jego miejsce
5. Powstają pary (i,j) elementów do zamiany
6. Dla każdej pary algorytm próbuje umieścić element i na pozycji zajmowanej przez j w drugim rodzicu
7. Jeśli nie jest to możliwe (pozycja już zajęta), sprawdzany jest element na tej pozycji i proces jest powtarzany
8. Pozostałe pozycje są wypełniane elementami z drugiego rodzica

[4] https://sound.eti.pg.gda.pl/student/isd/isd03-algorytmy_genetyczne.pdf

❖ **Operator Order Crossover** (OX, Davis, 1985):

Jest to operator, którego główną ideą jest zachowanie względnej kolejności miast w cyklu. Działa następująco:

1. Losowo wybierany jest segment, który jest kopiowany bezpośrednio z pierwszego rodzica do potomka
2. Pozostałe pozycje są wypełniane elementami w kolejności, w jakiej występują w drugim rodzicu
3. Podczas wypełniania pomijane są elementy, które zostały już skopiowane z pierwszego rodzica
4. Wypełnianie rozpoczyna się od pozycji za skopiowanym segmentem, z możliwością zawinięcia na początek permutacji
5. W ten sposób zachowywana jest względna kolejność elementów z drugiego rodzica, przy jednoczesnym zachowaniu fragmentu z pierwszego rodzica

Kluczową różnicą między tymi operatorami jest to, że PMX skupia się na zachowaniu bezwzględnych pozycji elementów, podczas gdy OX koncentruje się na zachowaniu względnej kolejności elementów w permutacji.

1.2.4.3 Mutacja

- losowa zmiana w genotypie, zwiększająca różnorodność populacji. W algorytmach genetycznych symuluje się ten proces z niskim prawdopodobieństwem, aby uniknąć nadmiernej losowości, która mogłaby osłabić utrwalenie korzystnych zmian. Dzięki mutacji algorytm może poprawić zdolność do znajdowania optymalnych rozwiązań.

1.2.5 Strategia wymiany pokoleń

- określa sposób zastępowania starej populacji przez nową (np. model generacyjny, model stacjonarny).

1.2.6 Kryterium zatrzymania

- warunek określający, kiedy algorytm powinien zakończyć działanie (np. osiągnięcie określonej liczby iteracji, brak poprawy najlepszego rozwiązania przez określoną liczbę pokoleń).

1.2.7 Parametry sterujące [5]

- Wielkość populacji
- Prawdopodobieństwo krzyżowania
- Prawdopodobieństwo mutacji
- Warunek zatrzymania (np. liczba pokoleń, kryterium zbieżności)

2. Opis klas

Wszystkie funkcje są zaimplementowane w klasie genetic_tsp, która obejmuje opisane poniżej metody. Konstruktor klasy inicjalizuje wartości takie jak timeLimit oraz optimalDistance, zależnie od rozmiaru wejściowego pliku. Dodatkowo generuje początkową populację, tworzoną losowo w celu rozpoczęcia algorytmu genetycznego.

[5] https://www.researchgate.net/publication/261135110_Analysis_of_the_impact_of_parameters_values_on_the_Genetic_Algorithm_for_TSP

```
// Zainicjuj populację początkową
population.clear();
for (int i = 0; i < population_size; i++) {
    std::vector<int> chromosome(numCities);
    for (int j = 0; j < numCities; j++) {
        chromosome[j] = j;
    }
    std::shuffle(first: chromosome.begin(), last: chromosome.end(), [>>] gen);
    population.push_back(chromosome);
}
```

Rysunek 1 Fragment kodu przedstawiający generowanie populacji początkowej

❖ Funkcja run()

Funkcja int genetic_tsp::run() implementuje algorytm genetyczny dla problemu komiwojażera (TSP). Na początku inicjalizowane są zmienne: czas rozpoczęcia, najlepsze rozwiązanie (bestSolution), oraz jego koszt (bestDistance jako nieskończoność).

W pętli działającej przez określony czas (timeLimit) populacja jest aktualizowana:

1. **Selekcja elity**: Najlepsze 10% osobników z obecnej populacji jest kopiowane do nowej.
2. **Nowe losowe rozwiązania**: Generuje się 40% populacji jako losowe permutacje miast.
3. **Krzyżowanie i mutacja:** Reszta populacji tworzona jest poprzez krzyżowanie (OX lub PMX) i mutację (swap lub inversion).

Po każdej iteracji aktualizowane są bestSolution i bestDistance, jeśli znaleziono lepsze rozwiązanie. Wyniki obejmujące czas, błąd względny, i najlepsze rozwiązanie są rejestrowane. Funkcja zwraca najlepszy znaleziony dystans.

❖ Mutacje

Funkcja mutationSwap wprowadza mutację w chromosomie poprzez zamianę miejscami dwóch losowo wybranych genów (miast).

1. **Losowa decyzja o mutacji:** Jeśli losowa wartość z przedziału [0.0,1.0)[0.0, 1.0)[0.0,1.0) jest mniejsza niż mutation_rate, następuje mutacja.
2. **Wybór pozycji:** Dwie losowe pozycje w chromosomie są wybrane przy użyciu rozkładu jednostajnego.
3. **Zamiana genów:** Geny na tych pozycjach są zamieniane miejscami za pomocą std::swap.

```
void genetic_tsp::mutationSwap(std::vector<int> &chromosome) {
    std::uniform_real_distribution<> dis(a: 0.0, b: 1.0);
    if (dis([&] gen) < mutation_rate) {
        std::uniform_int_distribution<> posDis(a: 0, b: numCities - 1);
        int pos1 = posDis([&] gen);
        int pos2 = posDis([&] gen);
        std::swap(a: [&] chromosome[pos1], b: [&] chromosome[pos2]);
    }
}
```

Rysunek 2 Fragment kodu przedstawiający funkcję z mutationSwap

Funkcja mutationInversion wprowadza mutację poprzez odwrócenie kolejności genów w losowym fragmencie chromosomu.

1. **Losowa decyzja o mutacji:** Mutacja zachodzi, jeśli losowa wartość z przedziału [0.0,1.0)[0.0, 1.0)[0.0,1.0) jest mniejsza niż mutation_rate.
2. **Wybór zakresu:** Za pomocą rozkładu jednostajnego losowane są dwie pozycje w chromosomie, określające początek (start) i koniec (end) zakresu do odwrócenia. Jeśli start > end, pozycje są zamieniane.

3. **Odwrócenie fragmentu:** Elementy w wybranym zakresie są odwracane za pomocą std::reverse.

```
void genetic_tsp::mutationInversion(std::vector<int> &chromosome) {
    std::uniform_real_distribution<> dis( a: 0.0,  b: 1.0);
    if (dis( [&] gen) < mutation_rate) {
        std::uniform_int_distribution<> posDis( a: 0,  b: numCities - 1);
        int start = posDis( [&] gen);
        int end = posDis( [&] gen);
        if (start > end) std::swap( a: [&] start,  b: [&] end);

        std::reverse( first: chromosome.begin() + start,
                      last: chromosome.begin() + end + 1);
    }
}
```

Rysunek 3 Fragment kodu przedstawiający funkcję z mutationInversion

❖ Krzyżowanie

Funkcja crossoverOX (Order Crossover) tworzy potomka, zachowując porządek genów z jednego rodzica oraz uzupełniając go genami z drugiego rodzica w odpowiedniej kolejności.

1. **Losowanie segmentu**: Wybierany jest losowy zakres pozycji [start,end][start, end][start,end] w chromosomie.
2. **Kopiowanie segmentu**: Elementy z zakresu [start,end][start, end][start,end] są kopiowane z parent1 do odpowiadających pozycji w potomku.
3. **Oznaczenie użytych genów**: Tablica used zaznacza, które geny zostały już wykorzystane.
4. **Uzupełnianie brakujących genów**: Pozostałe pozycje są wypełniane genami z parent2, zachowując ich kolejność i pomijając już wykorzystane.

Potomek zawiera geny obu rodziców w logicznej kolejności, zgodnej z porządkiem miast w problemie TSP.

```
std::vector<int> genetic_tsp::crossoverOX(const std::vector<int> &parent1, const std::vector<int> &parent2) {
    std::uniform_int_distribution<> dis( a: 0,  b: numCities - 1);
    int start = dis( [&] gen);       // losujemy miasta
    int end = dis( [&] gen);
    if (start > end) std::swap( a: [&] start,  b: [&] end);

    std::vector<int> child(numCities,  value: -1);
    std::vector<bool> used(numCities,  value: false);

    // Kopiowanie segmentu z pierwszego rodzica
    for (int i = start; i <= end; i++) {
        child[i] = parent1[i];
        used[parent1[i]] = true;
    }
    // Wypełnianie pozostałych pozycji
    int currentPos = (end + 1) % numCities;
    int parent2Pos = (end + 1) % numCities;

    while (currentPos != start) {
        while (used[parent2[parent2Pos]]) {
            parent2Pos = (parent2Pos + 1) % numCities;
        }
        child[currentPos] = parent2[parent2Pos];
        used[parent2[parent2Pos]] = true;
        currentPos = (currentPos + 1) % numCities;
    }
    return child;
}
```

Rysunek 4 Fragment kodu przedstawiający funkcję krzyżowania OX

Funkcja crossoverPMX (Partially Matched Crossover) tworzy potomka, przenosząc geny z jednego rodzica i mapując konflikty, aby zachować prawidłowy chromosom.

1. **Losowanie segmentu:** Wyznaczany jest losowy zakres pozycji [start,end][start, end][start,end].
2. **Tworzenie mapowania:** Dla każdej pozycji w zakresie [start,end][start, end][start,end] z parent1 i parent2 tworzona jest mapująca relacja pomiędzy genami obu rodziców.
3. **Kopiowanie segmentu:** Geny z zakresu są kopiowane z parent1 do odpowiadających pozycji w potomku.
4. **Rozwiązywanie konfliktów:** Dla pozycji spoza zakresu [start,end][start, end][start,end], konfliktowe geny są zastępowane zgodnie z mapowaniem, aby zapewnić unikalność każdego genu.

Potomek zachowuje strukturę genów rodziców, minimalizując konflikty i umożliwiając lepsze przenoszenie cech między rodzicami.

```
std::vector<int> genetic_tsp::crossoverPMX(const std::vector<int> &parent1, const std::vector<int> &parent2) {
    std::uniform_int_distribution<> dis(a: 0, b: numCities - 1);
    int start = dis([&] gen);
    int end = dis([&] gen);
    if (start > end) std::swap(a: [&] start, b: [&] end);

    std::vector<int> child(parent2);
    std::vector<int> mapping(numCities, value: -1);

    // tworzenie mapowania miast w środku
    for (int i = start; i <= end; i++) {
        int value1 = parent1[i];
        int value2 = parent2[i];
        mapping[value1] = value2;   // miasto1 -> miasto2
        mapping[value2] = value1;   // miasto2 -> miasto1
        child[i] = parent1[i];  // podmiana
    }
    // rozwiązywanie konfliktów przy mapowaniu
    for (int i = 0; i < numCities; i++) {
        if (i < start || i > end) { // tylko dla pozycji poza środkiem
            int current = child[i];
            while (mapping[current] != -1) {
                current = mapping[current];
            }
            child[i] = current;
        }
    }
    return child;
}
```

Rysunek 5 Fragment kodu przedstawiający funkcję krzyżowania PMX

❖ Selekcja

Funkcja tournamentSelection wybiera chromosom metodą turniejową, losując 3 osobniki z populacji i porównując ich fitness. Najlepszy (o najmniejszym fitnessie) zostaje wybrany i zwrócony. Dzięki temu zwiększa się szansa wyboru lepszego rozwiązania, zachowując element losowości.

```
std::vector<int> genetic_tsp::tournamentSelection() {
    const int tournamentSize = 3;
    std::uniform_int_distribution<> dis(a: 0, b: population_size - 1);

    int bestFitness = std::numeric_limits<double>::infinity();
    int bestIndex = -1;

    for (int i = 0; i < tournamentSize; i++) {
        int idx = dis([&] gen);
        int fitness = calculateFitness(population[idx]);
        if (fitness < bestFitness) {
            bestFitness = fitness;
            bestIndex = idx;
        }
    }

    return population[bestIndex];
}
```

Rysunek 6 Fragment kodu przedstawiający funkcję tournamentSelection

- **Funkcja fitness**

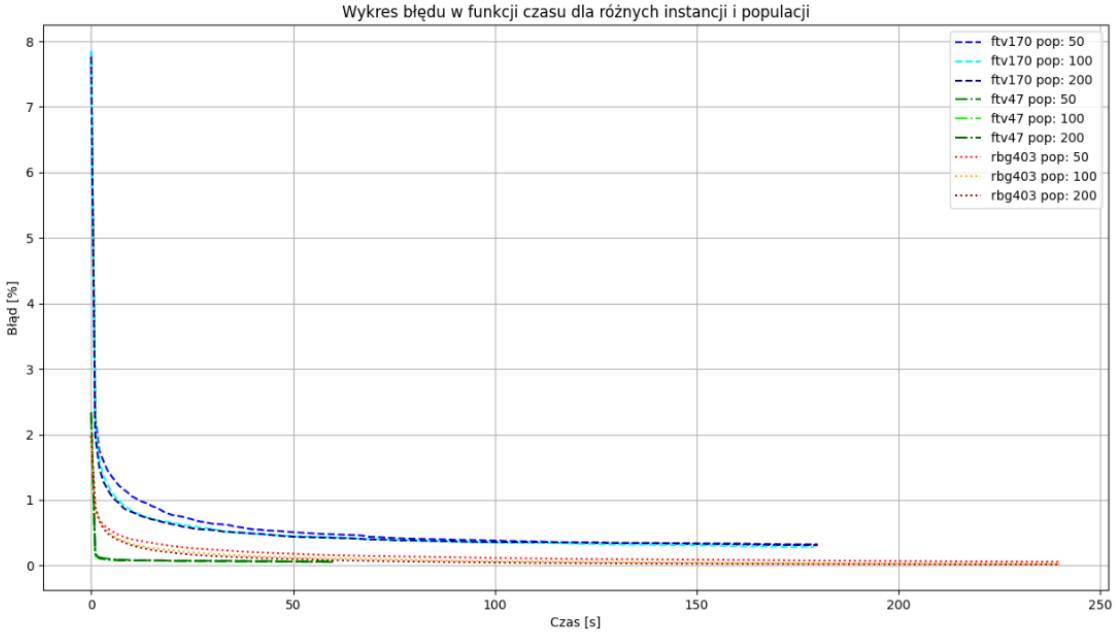
Funkcja calculateFitness oblicza całkowity dystans trasy w chromosomie dla problemu TSP. Iteruje przez wszystkie miasta, sumując dystanse między kolejnymi miastami, a na końcu uwzględnia powrót z ostatniego miasta do pierwszego. Dystanse pobierane są z macierzy wag za pomocą matrix.getWeight. Funkcja zwraca łączny koszt trasy, gdzie mniejsza wartość oznacza lepsze rozwiązanie.

```
int genetic_tsp::calculateFitness(const std::vector<int> &chromosome) const {
    int totalDistance = 0;
    for (int i = 0; i < numCities; i++) {
        int city1 = chromosome[i];
        int city2 = chromosome[(i + 1) % numCities];
        totalDistance += matrix.getWeight(i: city1, j: city2);
    }
    return totalDistance;
}
```

Rysunek 7 Fragment kodu przedstawiający funkcję calculateFitness

3. Wyniki eksperymentu

3.1. Badanie wpływu wielkości populacji



Rysunek 8 Wykres zależności rozmiaru populacji dla trzech plików
Źródło: Opracowanie własne

Przedstawiony wykres ilustruje zależność pomiędzy wielkością populacji w algorytmie genetycznym a jakością uzyskanych rozwiązań dla trzech różnych instancji problemu. Badania przeprowadzono dla macierzy o zróżnicowanych rozmiarach: ftv47 (47 wierzchołków), ftv170 (170 wierzchołków) oraz rbg403 (403 wierzchołki).

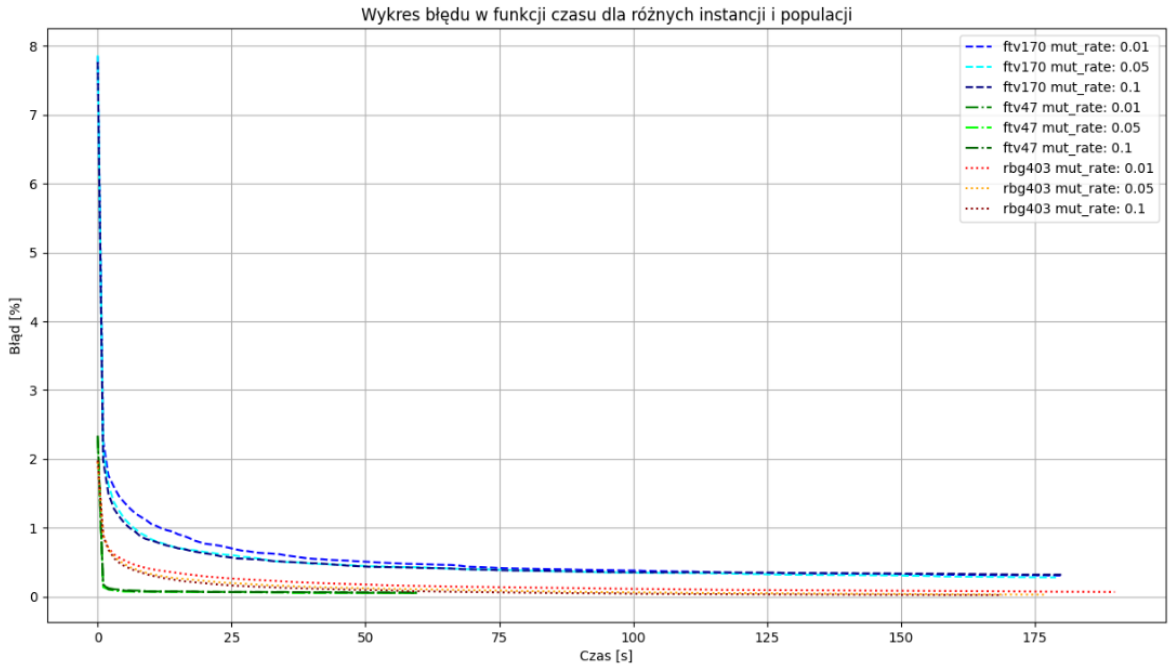
Dla najmniejszej macierzy ftv47 algorytm bardzo szybko osiąga wysokiej jakości rozwiązania, niezależnie od wielkości populacji. Błąd początkowy obniża się gwałtownie w pierwszych sekundach działania algorytmu, stabilizując się na bardzo niskim poziomie, wynoszącym około 0,1%. Wyniki te wskazują na dużą skuteczność algorytmu w przypadku problemów o niewielkiej skali.

W przypadku macierzy ftv170 odnotowano najwyższy błąd początkowy, sięgający 8%. Proces optymalizacji trwa dłużej w porównaniu z macierzą ftv47, a końcowy błąd jest nieco wyższy. Różnice w jakości rozwiązań pomiędzy populacjami o różnej wielkości są niewielkie, przy czym większe populacje (200 osobników) generują minimalnie lepsze rezultaty.

Największa z badanych macierzy, rbg403, dostarcza interesujących obserwacji. Mimo znacznej liczby wierzchołków, algorytm osiąga bardzo dobre wyniki. Początkowy błąd utrzymuje się na umiarkowanym poziomie około 2%, zaś końcowy błąd jest zbliżony do wyników uzyskanych dla najmniejszej instancji ftv47.

Analiza wykresu wskazuje, że rozmiar problemu nie jest jedynym determinantem jakości otrzymywanych rozwiązań. Zwiększenie wielkości populacji ma relatywnie niewielki wpływ na końcową jakość rozwiązań, choć może nieznacznie przyspieszać zbieżność algorytmu. Wyniki sugerują, że dla badanych instancji nawet mniejsze populacje (50 osobników) są wystarczające do uzyskania dobrych rozwiązań, dlatego do dalszych badań wybrano rozmiar 100 do dalszych badań współczynników.

3.2. Badanie wpływu wielkości współczynnika mutacji



Rysunek 9 Wykres zależności wielkości współczynnika mutacji dla trzech plików
Źródło: opracowanie własne

Wykres ilustruje wpływ współczynnika mutacji (0.01, 0.05, 0.1) na efektywność algorytmu genetycznego w przypadku trzech instancji problemu wspomnianych w poprzednim punkcie.

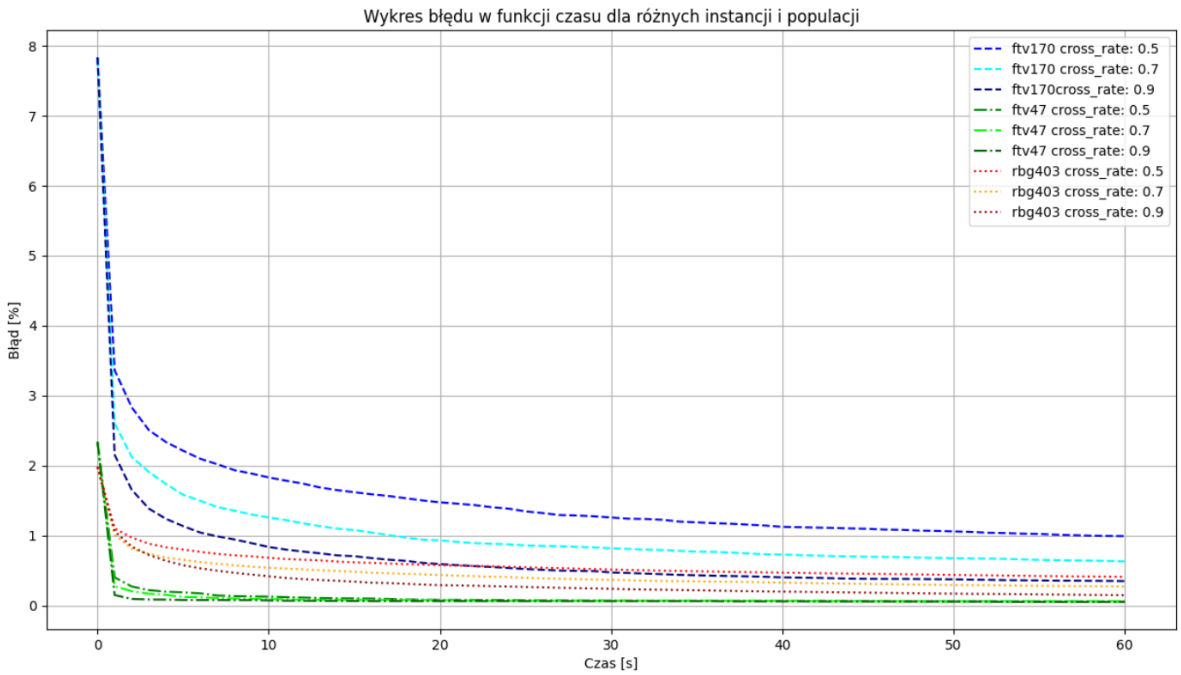
Dla najmniejszej instancji ftv47 algorytm charakteryzuje się bardzo szybkim procesem zbieżności. Niezależnie od zastosowanego współczynnika mutacji, końcowy błąd osiąga wartość około 0,1%. Różnice pomiędzy wynikami dla poszczególnych wartości współczynnika są minimalne, co wskazuje na niską wrażliwość algorytmu na tę zmienną w przypadku mniejszych problemów.

Dla średniej wielkości instancji ftv170 odnotowano najwyższy błąd początkowy, wynoszący około 8%. Współczynnik mutacji równy 0.01 generuje nieznacznie gorsze wyniki w porównaniu do wartości 0.05 oraz 0.1. Ostatecznie błąd stabilizuje się na poziomie około 0,4%, przy czym większe współczynniki mutacji wykazują lepszą efektywność.

Dla największej z badanych instancji rbg403 algorytm osiąga umiarkowany błąd początkowy, wynoszący około 2%. Proces zbieżności przebiega efektywnie, a końcowy błąd jest porównywalny do wyników uzyskanych dla najmniejszej instancji, oscylując wokół wartości 0,1%. Zastosowanie większych współczynników mutacji (0.05 i 0.1) daje nieznacznie lepsze rezultaty w porównaniu do współczynnika 0.01.

Wpływ współczynnika mutacji na jakość końcowych rozwiązań jest stosunkowo niewielki, szczególnie dla małych instancji problemu. Jednak dla większych instancji (ftv170 i rbg403) wyższe wartości współczynnika mutacji (0.05 i 0.1) mogą prowadzić do minimalnie lepszych wyników, wskazując na ich potencjalną przydatność w bardziej złożonych przypadkach.

3.3. Badanie wpływu wielkości współczynnika krzyżowania



Rysunek 10 Wykres zależności wielkości współczynnika krzyżowania dla trzech plików
Źródło: opracowanie własne

Wykres przedstawia wpływ współczynnika krzyżowania (0.5, 0.7, 0.9) na efektywność algorytmu genetycznego w kontekście trzech znanych nam już instancji problemu.

Dla najmniejszej instancji ftv47 algorytm wykazuje bardzo szybką zbieżność i niski błąd końcowy, wynoszący około 0,1%. Różnice w wynikach uzyskanych dla poszczególnych wartości współczynnika krzyżowania są minimalne, co wskazuje na niewielką wrażliwość algorytmu na tę zmienną w przypadku problemów o mniejszej skali.

Średniej wielkości instancja ftv170 jest najbardziej wrażliwa na zmiany współczynnika krzyżowania. Wartość 0.9 generuje najlepsze wyniki, zapewniając szybszą zbieżność i niższy końcowy błąd (około 0,4%). Współczynnik 0.5 prowadzi do wolniejszej zbieżności i wyższego błędu końcowego, wynoszącego około 1%. Różnice w efektywności pomiędzy analizowanymi wartościami współczynnika są tutaj najbardziej zauważalne.

Dla największej instancji rbg403 współczynnik krzyżowania ma umiarkowany wpływ na działanie algorytmu. Wyższe wartości współczynnika (0.7 i 0.9) zapewniają lepszą zbieżność oraz niższy końcowy błąd w porównaniu do współczynnika 0.5. Niemniej jednak różnice pomiędzy analizowanymi wartościami są mniejsze niż w przypadku ftv170.

Wyższe współczynniki krzyżowania (0.7–0.9) prowadzą do lepszych rezultatów, szczególnie w przypadku większych i bardziej złożonych instancji problemu, takich jak ftv170 i rbg403. Efekt ten jest

najbardziej wyraźny dla instancji ftv170, gdzie odpowiedni dobór współczynnika krzyżowania może znacząco poprawić efektywność algorytmu.

4. Wnioski

Przeprowadzone badania analizowały wpływ różnych parametrów algorytmu genetycznego – wielkości populacji, współczynnika mutacji oraz współczynnika krzyżowania – na jakość uzyskiwanych rozwiązań dla trzech instancji problemu: ftv47 (47 wierzchołków), ftv170 (170 wierzchołków) oraz rbg403 (403 wierzchołki). Wyniki wskazują na istotne różnice w działaniu algorytmu w zależności od analizowanych parametrów i rozmiaru problemu.

Dla najmniejszej instancji (ftv47) algorytm bardzo szybko osiąga wysoką jakość rozwiązań, niezależnie od wielkości populacji czy wartości współczynników mutacji i krzyżowania. Końcowy błąd stabilizuje się na bardzo niskim poziomie około 0,1%, a różnice w wynikach są minimalne. Wskazuje to na niską wrażliwość algorytmu na zmiany parametrów w przypadku problemów o niewielkiej skali.

Średniej wielkości instancja (ftv170) charakteryzuje się najwyższym błędem początkowym, wynoszącym około 8%, oraz dłuższym procesem zbieżności. Wielkość populacji ma ograniczony wpływ na końcową jakość rozwiązań, jednak większe populacje (200 osobników) generują minimalnie lepsze rezultaty. W przypadku współczynnika mutacji, wyższe wartości (0.05 i 0.1) prowadzą do nieco lepszych wyników niż 0.01. Najbardziej widoczny wpływ odnotowano dla współczynnika krzyżowania, gdzie wartość 0.9 zapewnia szybszą zbieżność i najniższy końcowy błąd (0,4%), podczas gdy współczynnik 0.5 prowadzi do wyższego błędu końcowego (1%).

Największa instancja (rbg403) wykazuje umiarkowany błąd początkowy, wynoszący około 2%, a końcowy błąd stabilizuje się na poziomie zbliżonym do najmniejszej instancji (0,1%). Wpływ wielkości populacji, współczynnika mutacji i współczynnika krzyżowania na jakość rozwiązań jest zauważalny, lecz stosunkowo niewielki. Wyższe wartości współczynnika mutacji (0.05 i 0.1) oraz krzyżowania (0.7 i 0.9) zapewniają lepsze wyniki, szczególnie w zakresie szybkości zbieżności.

Podsumowując, większe wartości współczynnika mutacji (0.05–0.1) i krzyżowania (0.7–0.9) zazwyczaj prowadzą do lepszych wyników dla bardziej złożonych instancji problemu. Rozmiar populacji odgrywa istotną rolę w przyspieszaniu zbieżności, ale nawet mniejsze populacje (50 osobników) są wystarczające do uzyskania dobrych rozwiązań w większości przypadków. Wyniki wskazują, że odpowiedni dobór parametrów może znacząco poprawić efektywność algorytmu, szczególnie w przypadku bardziej wymagających problemów, takich jak ftv170 i rbg403.

5. Kod źródłowy

Zostanie załączony wraz ze sprawozdaniem.

Bibliografia:

1. https://kotrak.com/pl/blog/algorytmy-genetyczne-w-systemach-it-i-przemysle/
2. https://pl.wikipedia.org/wiki/Algorytm_genetyczny
3. http://staff.iiar.pwr.wroc.pl/dariusz.banasiak/si/SI_wyklad10.pdf
4. https://sound.eti.pg.gda.pl/student/isd/isd03-algorytmy_genetyczne.pdf
5. https://www.researchgate.net/publication/261135110_Analysis_of_the_impact_of_parameters_values_on_the_Genetic_Algorithm_for_TSP